



**BHARATI VIDYAPEETH'S
INSTITUTE OF COMPUTER APPLICATION & MANAGEMENT**

(Affiliated to Guru Gobind Singh Indraprastha University,

Approved by AICTE, New Delhi)

Python Programming Lab (MCA- 166) Practical File

Submitted To:

Dr. Saumya Bansal
(Assistant Professor)

Submitted By:

Piyush Goel (09311604425)
MCA 2nd Sem, Sec B

INDEX

S.No.	Problem Description	Date	Sign.
P1	Implement Python Script to generate first N natural numbers.		
P2	By considering the terms in the Fibonacci sequence whose values do not exceed 1000, find the sum of the even-valued terms.		
P3	Write a program which makes use of function to display all such numbers which are divisible by 7 but are not a multiple of 5, between 1000 and 2000.		
P4	Write a function cumulative_product to compute cumulative product of a list of numbers.		
P5	Write a function reverse to reverse a list. Without using the reverse function.		
P6	Write a function that finds the sum of first n terms of following series: $1 - x^2 / 2! + x^4 / 4! - x^6 / 6! + \dots x^n / n!$		
P7	With a given tuple (1, 2, 3, 4, 5, 6, 7, 8, 9, 10), write a program to print the first half values in one line and the last half values in one line		
P8	Implement a python script to check the element is in the list or not by using Linear search & Binary search.		
P9	Write a program to count the numbers of characters in the string and store them in a dictionary data structure.		
P10	Every time a vote is cast the name of the candidate is appended to the data structure. Print the names of candidates who received maximum vote in lexicographical order and if there is a tie print lexicographically smaller name.		
P11	Remove spaces from a string using recursion.		
P12	Write a Python class to reverse a string word by word : Input string : 'hello Python' Expected Output : 'Python hello'		

Python Programming Lab (MCA-166) Practical File

P13	Write a Python class which has two methods get_String and print_String. get_String accept a string from the user and print_String print the string in upper case.		
P14	Write a Python class named Circle constructed by a radius and two methods which will compute the area and the perimeter of a circle.		
P15	Implement a python script to arrange the elements in sorted order using Bubble, Selection, Insertion and Merge sorting techniques.		
P16	Create a menu driven program to perform various matrices operations.		
P17	Write a program to compare two files and display total number of lines in a file.		
P18	Write a program to compare two files and display total number of words in a file.		
P19	Draw different graphs and plots in Python using Matplotlib Library.		
P20	Write a python code for simple GUI calculator using Tk		
P21	Write a python program to perform various database operations (create, insert, delete, update)		

P1. Implement Python Script to generate first N natural numbers.

Solution:-

```
def generate_natural_numbers(n):  
    if n < 1:  
        print("Please enter a positive integer greater than 0.")  
        return []  
    natural_numbers = [i for i in range(1, n + 1)]  
    return natural_numbers
```

```
N = int(input("Enter the value of N: "))  
numbers = generate_natural_numbers(N)  
print("First", N, "natural numbers:", numbers)
```

Output:-

```
= RESTART: C:/Users/fitwi/AppData/Local/Programs/Python/Python314/practical.py =  
Enter the value of N: 5  
First 5 natural numbers: [1, 2, 3, 4, 5]
```

P2. By considering the terms in the Fibonacci sequence whose values do not exceed 1000, find the sum of the even-valued terms.

Solution:-

```
def even_fibonacci_sum(limit):
    a, b = 0, 1
    even_sum = 0
    while a <= limit:
        if a % 2 == 0:
            even_sum += a
        a, b = b, a + b
    return even_sum

limit = 1000
result = even_fibonacci_sum(limit)
print("Sum of even Fibonacci numbers up to", limit, "is:", result)
```

Output:-

```
= RESTART: C:/Users/fitwi/AppData/Local/Programs/Python/Python314/practical.py =
Sum of even Fibonacci numbers up to 1000 is: 798
|
```

P3. Write a program which makes use of function to display all such numbers which are divisible by 7 but are not a multiple of 5, between 1000 and 2000.

Solution:-

```
def divisible_by_7_not_5(start, end):
    result = []
    for number in range(start, end + 1):
        if number % 7 == 0 and number % 5 != 0:
            result.append(number)
    return result

start = 1000
end = 2000

numbers = divisible_by_7_not_5(start, end)
print("Numbers divisible by 7 but not by 5 between", start, "and", end, "are: \n", numbers)
```

Output:-

```
= RESTART: C:/Users/fitwi/AppData/Local/Programs/Python/Python314/practical.py =
Numbers divisible by 7 but not by 5 between 1000 and 2000 are:
[1001, 1008, 1022, 1029, 1036, 1043, 1057, 1064, 1071, 1078, 1092, 1099, 1106, 1113, 1127, 1134, 1141, 1148, 1162, 1169,
1176, 1183, 1197, 1204, 1211, 1218, 1232, 1239, 1246, 1253, 1267, 1274, 1281, 1288, 1302, 1309, 1316, 1323, 1337, 1344, 1
351, 1358, 1372, 1379, 1386, 1393, 1407, 1414, 1421, 1428, 1442, 1449, 1456, 1463, 1477, 1484, 1491, 1498, 1512, 1519, 15
26, 1533, 1547, 1554, 1561, 1568, 1582, 1589, 1596, 1603, 1617, 1624, 1631, 1638, 1652, 1659, 1666, 1673, 1687, 1694, 170
1, 1708, 1722, 1729, 1736, 1743, 1757, 1764, 1771, 1778, 1792, 1799, 1806, 1813, 1827, 1834, 1841, 1848, 1862, 1869, 1876
, 1883, 1897, 1904, 1911, 1918, 1932, 1939, 1946, 1953, 1967, 1974, 1981, 1988]
```

P4. Write a function `cumulative_product` to compute cumulative product of a list of numbers.

Solution:-

```
def cumulative_product(numbers):
    result = []
    product = 1
    for num in numbers:
        product *= num
        result.append(product)
    return result

nums = [2, 3, 4, 5]
print("Original list:", nums)
print("Cumulative product:", cumulative_product(nums))
```

Output:-

```
===== RESTART: C:/Users/rltwl/App
Original list: [2, 3, 4, 5]
Cumulative product: [2, 6, 24, 120]
|
```

P5. Define a function which generates Fibonacci series up to n numbers using RECURSION.

Solution:-

```
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
        seq = fibonacci(n - 1)
        seq.append(seq[-1] + seq[-2])
        return seq

n = 10
print(fibonacci(n))
```

Output:-

```
----- RESTART: C:/Users/11CWI/App
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

P6. Write a function that finds the sum of first n terms of following series: $1 - x^2 / 2! + x^4 / 4! - x^6 / 6! + \dots + x^n / n!$

Solution:-

```
import math
```

```
def series_sum(x, n):  
    result = 0  
    sign = 1  
    for i in range(0, n + 1, 2):  
        term = sign * (x ** i) / math.factorial(i)  
        result += term  
        sign *= -1  
    return result
```

```
x = 2
```

```
n = 10
```

```
print(f"sum of first {n} terms with x = {x} : ",series_sum(x, n))
```

```
===== RES  
Sum of first 10 terms with x = 2 : -0.4161552028218696  
> |
```

P7. With a given tuple (1, 2, 3, 4, 5, 6, 7, 8, 9, 10), write a program to print the first half values in one line and the last half values in one line.

Solution:-

```
t = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
mid = len(t) // 2
print(t[:mid])
print(t[mid:])
```

Output:-

```
===== RESTART: C:/Users/fitwi/AppData/Lo
(1, 2, 3, 4, 5)
(6, 7, 8, 9, 10)
|
```

P8. Implement a python script to check the element is in the list or not by using Linear search & Binary search.

Solution:-

```
def linear_search(list, target):
    for i in range(len(list)):
        if list[i] == target:
            return True
    return False
```

```
def binary_search(list, target):
    low = 0
    high = len(list) - 1
    while low <= high:
        mid = (low + high) // 2
        if list[mid] == target:
            return True
        elif list[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return False
```

```
list = [3, 1, 4, 5, 9, 2]
target = 5
print("List : ", list)
print(f"Linear Search to find target {target}:", linear_search(list, target))
list.sort()
print("\nList : ", list)
print(f"Binary Search to find target {target}:", binary_search(list, target))
```

```
List : [3, 1, 4, 5, 9, 2]
Linear Search to find target 5: True

Sorted List : [1, 2, 3, 4, 5, 9]
Binary Search to find target 5: True
>>
```

P9. Write a program to count the numbers of characters in the string and store them in a dictionary data structure.

Solution:-

```
def count_characters(s):
    d = {}
    for char in s:
        d[char] = d.get(char, 0) + 1
    return d
```

```
s = "hello world"
print(count_characters(s))
```

Output:-

```
===== RESTART: C:/Users/fitwi/AppData/Local/Programs/Python/Python39-64/Python.exe
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
>
```

P10. Every time a vote is cast the name of the candidate is appended to the data structure. Print the names of candidates who received maximum vote in lexicographical order and if there is a tie print lexicographically smaller name.

Solution:

```
from collections import Counter
```

```
def find_winner(votes):
    vote_count = Counter(votes)
    max_votes = max(vote_count.values())
    winners = [name for name, count in vote_count.items() if count == max_votes]
    return sorted(winners)[0]
```

```
votes = ['Alice', 'Bob', 'Alice', 'Eve', 'Bob', 'Eve', 'Eve']
print("votes :", votes)
print("Winner :", find_winner(votes))
```

```
===== RESTART: C:/Users/fitwi/AppData/Local/Programs/Python/Python39-64/Python.exe
Votes : ['Alice', 'Bob', 'Alice', 'Eve', 'Bob', 'Eve', 'Eve']
Winner : Eve
>>
```

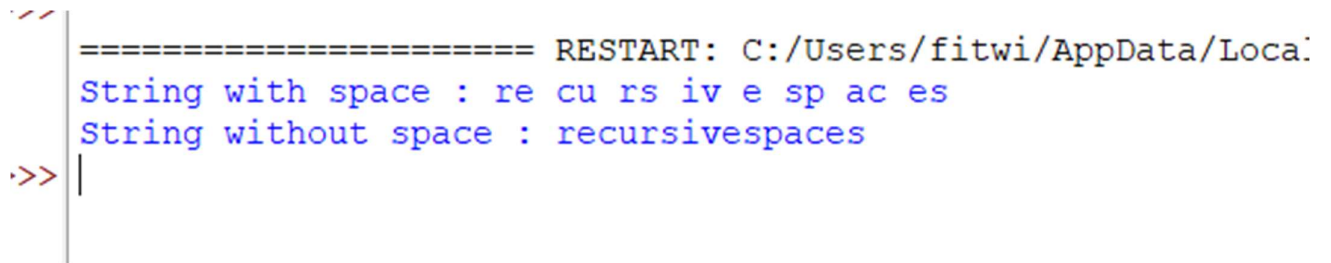
P11. Remove spaces from a string using recursion.

Solution:-

```
def remove_spaces(s):
    if not s:
        return ""
    if s[0] == " ":
        return remove_spaces(s[1:])
    else:
        return s[0] + remove_spaces(s[1:])

s = "re cu rs iv e sp ac es"
print("String with space : ", s)
print("String without space : ", remove_spaces(s))
```

Output:-

A screenshot of a Python IDE window. The title bar shows the file path: C:/Users/fitwi/AppData/Local... The editor contains the following text: ===== RESTART: C:/Users/fitwi/AppData/Local... String with space : re cu rs iv e sp ac es String without space : recursivespaces. The prompt '>>' is visible on the left side of the editor.

```
===== RESTART: C:/Users/fitwi/AppData/Local...
String with space : re cu rs iv e sp ac es
String without space : recursivespaces
>>
```

P12. Write a Python class to reverse a string word by word : Input string : 'hello Python'
Expected Output : 'Python hello'

Solution:-

```
class ReverseWords:
    def __init__(self, text):
        self.text = text

    def reverse(self):
        return ' '.join(self.text.split()[::-1])

s = ReverseWords('hello Python')
print("String : ", s.text)
print("Reverse String : ",s.reverse())
```

Output:-

```
String : hello Python
Reverse String : Python hello
> |
```

P13. Write a Python class which has two methods get_String and print_String. get_String accept a string from the user and print_String print the string in upper case.

Solution:-

```
class StringManipulation:
    def __init__(self):
        self.user_string = ""

    def get_String(self):
        self.user_string = input("Enter a string: ")

    def print_String(self):
        print(self.user_string.upper())

obj = StringManipulation()
obj.get_String()
obj.print_String()
```

```
-----
Enter a string: python programming
PYTHON PROGRAMMING
>> |
```

P14. Write a Python class named Circle constructed by a radius and two methods which will compute the area and the perimeter of a circle.

Solution:-

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius

c = Circle(5)
print("Area of circle : ",c.area())
print("Perimeter of circle : ",c.perimeter())
```

Output:-

```
Area of circle : 78.53981633974483
Perimeter of circle : 31.41592653589793
```

P15. Implement a python script to arrange the elements in sorted order using Bubble, Selection, Insertion and Merge sorting techniques.

Solution:

Bubble Sort

```
def bubble_sort(arr):  
    a = arr.copy()  
    n = len(a)  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if a[j] > a[j+1]:  
                a[j], a[j+1] = a[j+1], a[j]  
    return a
```

Selection Sort

```
def selection_sort(arr):  
    a = arr.copy()  
    n = len(a)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if a[j] < a[min_idx]:  
                min_idx = j  
        a[i], a[min_idx] = a[min_idx], a[i]  
    return a
```

Insertion Sort

```
def insertion_sort(arr):  
    a = arr.copy()  
    for i in range(1, len(a)):
```

```

    key = a[i]

    j = i - 1

    while j >= 0 and key < a[j]:
        a[j+1] = a[j]
        j -= 1
    a[j+1] = key

    return a

# Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])

```

```
        result.extend(right[j:])

    return result

# main
arr = [64, 34, 25, 12, 22, 11, 90]

print("Original list:", arr)
print("Bubble Sort:", bubble_sort(arr))
print("Selection Sort:", selection_sort(arr))
print("Insertion Sort:", insertion_sort(arr))
print("Merge Sort:", merge_sort(arr))

=====
Original list: [64, 34, 25, 12, 22, 11, 90]
Bubble Sort: [11, 12, 22, 25, 34, 64, 90]
Selection Sort: [11, 12, 22, 25, 34, 64, 90]
Insertion Sort: [11, 12, 22, 25, 34, 64, 90]
Merge Sort: [11, 12, 22, 25, 34, 64, 90]
```

P16. Create a menu driven program to perform various matrices operations.

Solution:-

```
def add_matrix(A, B):  
    result = []  
    for i in range(len(A)):  
        row = []  
        for j in range(len(A[0])):  
            row.append(A[i][j] + B[i][j])  
        result.append(row)  
    return result  
  
def subtract_matrix(A, B):  
    result = []  
    for i in range(len(A)):  
        row = []  
        for j in range(len(A[0])):  
            row.append(A[i][j] - B[i][j])  
        result.append(row)  
    return result  
  
def multiply_matrix(A, B):  
    result = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]  
  
    for i in range(len(A)):  
        for j in range(len(B[0])):  
            for k in range(len(B)):  
                result[i][j] += A[i][k] * B[k][j]  
    return result  
  
def transpose_matrix(A):  
    result = []  
    for j in range(len(A[0])):  
        row = []
```

```
        for i in range(len(A)):
            row.append(A[i][j])
        result.append(row)
    return result

def print_matrix(M):
    for row in M:
        print(row)

# input matrix
def input_matrix(rows, cols):
    matrix = []
    for i in range(rows):
        row = list(map(int, input().split()))
        matrix.append(row)
    return matrix

while True:
    print("\nMatrix Operations Menu")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Transpose")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        r, c = map(int, input("Enter rows and cols: ").split())
        print("Enter matrix A:")
        A = input_matrix(r, c)
        print("Enter matrix B:")
```

```
B = input_matrix(r, c)

print("Result:")
print_matrix(add_matrix(A, B))

elif choice == 2:

    r, c = map(int, input("Enter rows and cols: ").split())
    print("Enter matrix A:")
    A = input_matrix(r, c)
    print("Enter matrix B:")
    B = input_matrix(r, c)

    print("Result:")
    print_matrix(subtract_matrix(A, B))

elif choice == 3:

    r1, c1 = map(int, input("Enter rows and cols of A: ").split())
    print("Enter matrix A:")
    A = input_matrix(r1, c1)

    r2, c2 = map(int, input("Enter rows and cols of B: ").split())
    print("Enter matrix B:")
    B = input_matrix(r2, c2)

    if c1 != r2:

        print("Multiplication not possible")
    else:

        print("Result:")
        print_matrix(multiply_matrix(A, B))
```

```
elif choice == 4:

    r, c = map(int, input("Enter rows and cols: ").split())

    print("Enter matrix:")

    A = input_matrix(r, c)

    print("Transpose:")

    print_matrix(transpose_matrix(A))

elif choice == 5:

    break

else:

    print("Invalid choice")
```

```
Matrix Operations Menu
1. Addition
2. Subtraction
3. Multiplication
4. Transpose
5. Exit
Enter choice: 1
Enter rows and cols: 2 3
Enter matrix A:
2
3
Enter matrix B:
4
42
Result:
[6]
[45]

Matrix Operations Menu
1. Addition
2. Subtraction
3. Multiplication
4. Transpose
5. Exit
Enter choice:
```

P17. Write a program to compute the number of characters, words and lines in a file.

Solution:-

```
# create sample file
with open("sample.txt", "w") as f:
    f.write("Hello world\n")
    f.write("Python is easy\n")
    f.write("File handling example")

def count_file(filename):
    with open(filename, "r") as f:
        text = f.read()

    lines = text.split("\n")
    words = text.split()
    characters = len(text)

    print("Number of lines:", len(lines))
    print("Number of words:", len(words))
    print("Number of characters:", characters)

count_file("sample.txt")
=====
Number of lines: 3
Number of words: 8
Number of characters: 48
>> |
```

P18. Write a program to compare two files and display total number of lines in a file.

Solution:-

```
def compare_files(file1, file2):
    with open(file1, "r") as f1, open(file2, "r") as f2:
        lines1 = f1.readlines()
        lines2 = f2.readlines()

    print("Contents of", file1)
    print("".join(lines1))

    print("Contents of", file2)
    print("".join(lines2))

    print("Total lines in", file1, ":", len(lines1))
    print("Total lines in", file2, ":", len(lines2))

    if lines1 == lines2:
        print("Files are identical")
    else:
        print("Files are different")

compare_files("file1.txt", "file2.txt")
```

Output:-

```
Contents of file1.txt
hello
python
welcome
Contents of file2.txt
hello
python
world
Total lines in file1.txt : 3
Total lines in file2.txt : 3
Files are different
> |
```

P19. Draw different graphs and plots in Python using Matplotlib Library.

Solution:-

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)
z = np.cos(x)

plt.figure(figsize=(12, 10))

plt.subplot(2, 2, 1)
plt.plot(x, y, label='sin(x)', color='blue')
plt.title('Line Plot')
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.legend()

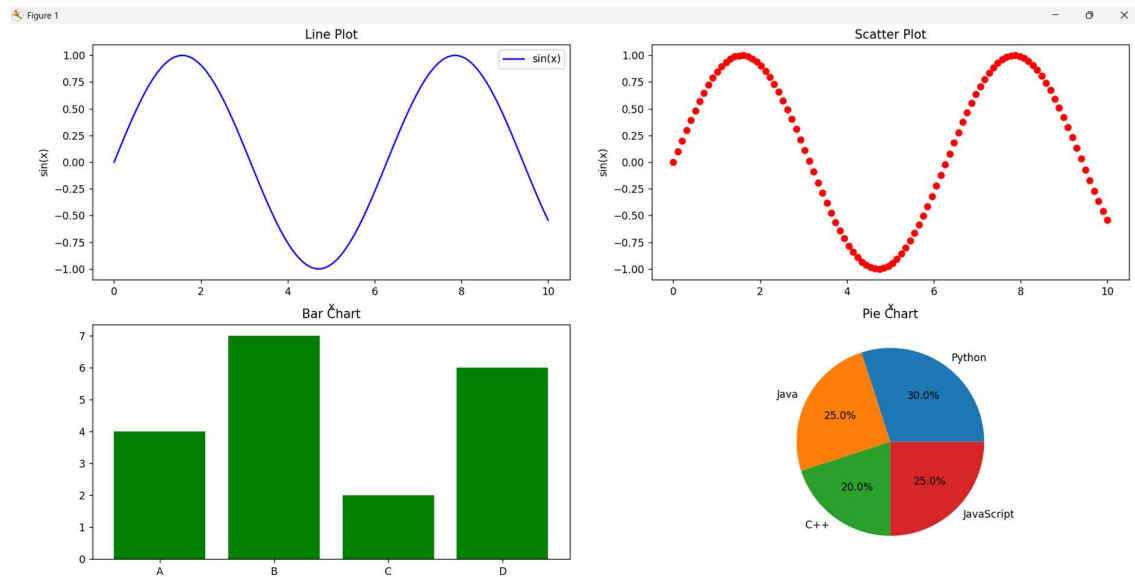
plt.subplot(2, 2, 2)
plt.scatter(x, y, color='red')
plt.title('Scatter Plot')
plt.xlabel('x')
plt.ylabel('sin(x)')

plt.subplot(2, 2, 3)
bars = ['A', 'B', 'C', 'D']
heights = [4, 7, 2, 6]
plt.bar(bars, heights, color='green')
plt.title('Bar Chart')

plt.subplot(2, 2, 4)
data = [30, 25, 20, 25]
labels = ['Python', 'Java', 'C++', 'JavaScript']
plt.pie(data, labels=labels, autopct='%1.1f%%')
plt.title('Pie Chart')

plt.tight_layout()
plt.show()
```

Output:-



P20. Write a python code for simple GUI calculator using Tk

Solution:

```
import tkinter as tk

# function to handle button click
def click(event):
    current = entry.get()
    entry.delete(0, tk.END)
    entry.insert(0, current + str(event.widget["text"]))

def clear():
    entry.delete(0, tk.END)

# evaluate expression
def equal():
    try:
        result = eval(entry.get())
        entry.delete(0, tk.END)
        entry.insert(0, str(result))
    except:
        entry.delete(0, tk.END)
        entry.insert(0, "Error")

root = tk.Tk()
root.title("Calculator")
root.geometry("300x350")

# entry box
entry = tk.Entry(root, font="Arial 18", justify="right")
```

```
entry.pack(fill="both", ipadx=8, pady=10, padx=10)
```

```
# buttons
```

```
buttons = [
```

```
    '7','8','9','/',
```

```
    '4','5','6','*',
```

```
    '1','2','3','-'
```

```
    '0','.','=','+'
```

```
]
```

```
frame = tk.Frame(root)
```

```
frame.pack()
```

```
row = 0
```

```
col = 0
```

```
for button in buttons:
```

```
    b = tk.Button(frame, text=button, font="Arial 14", width=5, height=2)
```

```
    b.grid(row=row, column=col, padx=5, pady=5)
```

```
    if button == "=":
```

```
        b.configure(command=equal)
```

```
    else:
```

```
        b.bind("<Button-1>", click)
```

```
    col += 1
```

```
    if col > 3:
```

```
        col = 0
```

```
row += 1
```

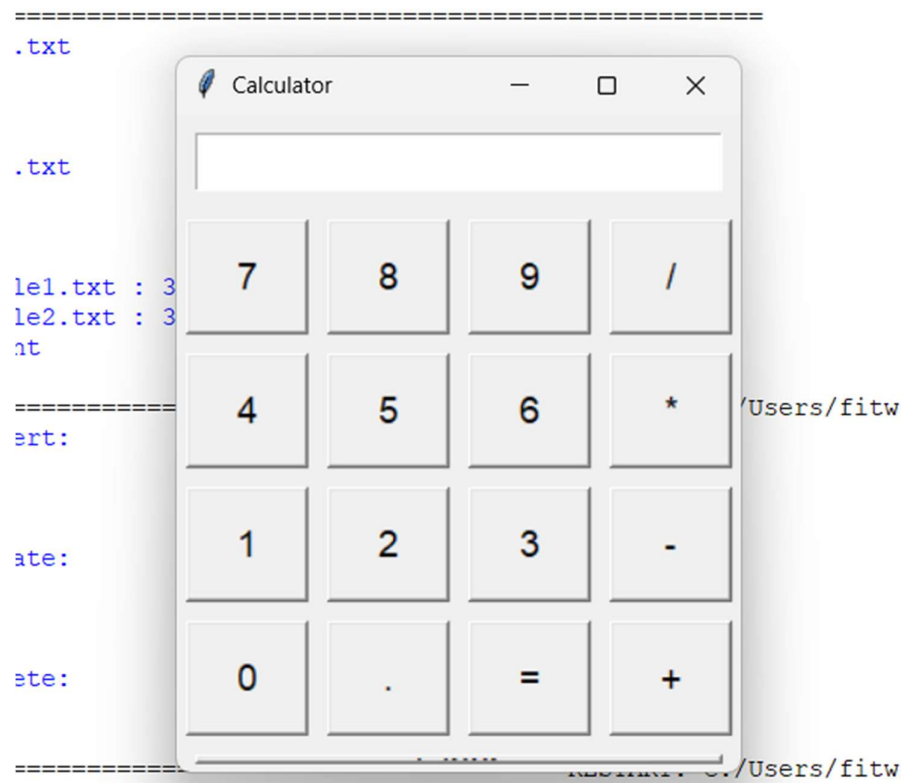
```
# clear button
```

```
clear_btn = tk.Button(root, text="Clear", font="Arial 14", command=clear)
```

```
clear_btn.pack(fill="both", padx=10, pady=5)
```

```
root.mainloop()
```

Output:



P21. Write a python program to perform various database operations (create, insert, delete, update)

Solution:-

```
import sqlite3

# connect to database (creates if not exists)
conn = sqlite3.connect("student.db")
cursor = conn.cursor()

# CREATE table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students(
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER
)
""")

cursor.execute("INSERT INTO students (name, age) VALUES (?, ?)", ("Alice", 21))
cursor.execute("INSERT INTO students (name, age) VALUES (?, ?)", ("Bob", 22))

conn.commit()

# DISPLAY data
print("Records after insert:")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)

cursor.execute("UPDATE students SET age = 23 WHERE name = 'Alice'")
conn.commit()

print("\nRecords after update:")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)

cursor.execute("DELETE FROM students WHERE name = 'Bob'")
conn.commit()
```

```
print("\nRecords after delete:")
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)
```

```
conn.close()
```

Output:-

```
=====
Records after insert:
(1, 'Alice', 21)
(2, 'Bob', 22)

Records after update:
(1, 'Alice', 23)
(2, 'Bob', 22)

Records after delete:
(1, 'Alice', 23)
|
```